

Day 12 复习总表：性能意识培养

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2 图片懒加载	项目卡片图片改为按需加载	<pre>html</pre>	• <code>loading="lazy"</code> 改变的是“什么时候加载”，不是“要不要加载” • 图片滚动到接近可视区域时才会真正发起请求，用 Network 面板可直接验证
Step 3 认识 Lighthouse	学会生成并解读性能报告	—	• 核心指标：FCP（首次内容渲染）、LCP（最大内容渲染，权重最高）、TBT（阻塞时长）、CLS（布局稳定性） • “Opportunities” 区域会直接给出可执行的优化建议
Step 4 首屏加载优化实践	用 <code>fetchpriority</code> 调整 CSS 加载优先级	<pre>html<link rel="stylesheet" href="styles.css" fetchpriority="high" /></pre>	• <code>fetchpriority="high"</code> 不改变“要不要等 CSS”，只影响“资源下载排队的优先级” • 性能优化很多时候是“缩短等待”而非“消除等待”
Step 5 优化前后对比	用同一套流程重新测量并对比	数据：LCP 2.0s → 1.0s，Performance 分数 99 → 99	• 分数在高分区间对时间提升不敏感，但真实秒数缩短是用户能感知的 • 专业流程：基线 → 干预（一次只改一处）→ 验证
Step 6 Git 打标签 + 推送	给当前进度打上版本标签	<pre>bashgit tag -a v0.3-week2-performance -m "...git pushgit push origin v0.3-week2-performance</pre>	• 标签一旦打上，永久指向那条 commit，不随后续提交移动 • 标签不会随普通 <code>git push</code> 自动同步，必须单独推送
Step 7 Code Review + 自评	5 道概念题检验理解	考察问题： 1. 懒加载改变的是什么 2. LCP 为何最重要 3. 分数没变为何仍算优化 4. 标签会不会随提交移动 5. 标签为何需单独推送	• 懒加载改变加载时机，不是加载与否 • LCP 是用户感知最直接的指标，权重也最高 • 分数不敏感 ≠ 没有真实提升，看绝对指标 • 标签固定指向某条 commit，不随后续提交移动 • 标签独立于分支历史，需专门推送

本日核心概念速查

概念	一句话记忆
<code>loading="lazy"</code>	改变“何时加载”，不改变“是否加载”

概念	一句话记忆
LCP	用户感知页面“基本加载完”的关键时刻，优先关注它
分数 vs 绝对指标	高分区间分数不敏感，看真实秒数才是硬道理
优化三步法	基线 → 干预（一次一变量）→ 验证
Git 标签	固定指向某条 commit 的“书签”，需要单独 <code>push</code> 才会同步到远程